

NAME :KRISHNAPRIYA P

COURSE CODE : CSA0613

REG NO: 192511279

COURSE NAME : DESIGN AND

ANALYSIS OF ALGORITHMS

CO3 - ASSESSMENT TOOL – 01

**Experiment: Evaluate Divide and Conquer Matrix Multiplication Performance Under
Varying Levels of Recursion Granularity**

Aim

To evaluate the performance of Divide and Conquer Matrix Multiplication by varying the recursion depth, measuring execution time for different splitting levels, and analyzing the effect of recursion granularity on computational overhead.

Objective

- To implement Divide and Conquer Matrix Multiplication.
- To measure execution time at different recursion depths.
- To compare the performance for various splitting levels.
- To analyze how recursion depth affects computational overhead.
- To identify the optimal recursion level for efficient execution.

Algorithm

- Start.
- Input two square matrices of size $N \times N$.
- Set the recursion depth (splitting depth).
- If the current depth reaches the specified limit or the matrix size becomes 1×1 , perform normal matrix multiplication.
- Otherwise:
 - Divide each matrix into four equal sub-matrices.
 - Recursively multiply the corresponding sub-matrices.
 - Combine the results into the final matrix.
 - Record the execution time.
- Repeat Steps 3–6 for different recursion depths.
- Compare execution times.
- Display the resulting matrix and performance analysis.
- Stop.

SOURCE CODE

```
main.py    Share Run Output
```

```
1 import time
2 import random
3
4 def add(A, B):
5     n = len(A)
6     return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
7
8 def matrix_multiply(A, B):
9     n = len(A)
10    C = [[0] * n for _ in range(n)]
11    for i in range(n):
12        for j in range(n):
13            for k in range(n):
14                C[i][j] += A[i][k] * B[k][j]
15    return C
16
17 def divide_conquer(A, B, depth, max_depth):
18     n = len(A)
19
20     if n == 1 or depth >= max_depth:
21         return matrix_multiply(A, B)
22
23     mid = n // 2
24
25     A11 = [row[:mid] for row in A[:mid]]
26     A12 = [row[mid:] for row in A[:mid]]
27     A21 = [row[:mid] for row in A[mid:]]
28     A22 = [row[mid:] for row in A[mid:]]
29
30     B11 = [row[:mid] for row in B[:mid]]
31     B12 = [row[mid:] for row in B[:mid]]
32     B21 = [row[:mid] for row in B[mid:]]
33     B22 = [row[mid:] for row in B[mid:]]
34
35     C11 = add(divide_conquer(A11, B11, depth+1, max_depth),
36               divide_conquer(A12, B21, depth+1, max_depth))
37
38     C12 = add(divide_conquer(A11, B12, depth+1, max_depth),
39               divide_conquer(A12, B22, depth+1, max_depth))
40
41     C21 = add(divide_conquer(A21, B11, depth+1, max_depth),
42               divide_conquer(A22, B21, depth+1, max_depth))
43
44     C22 = add(divide_conquer(A21, B12, depth+1, max_depth),
45               divide_conquer(A22, B22, depth+1, max_depth))
46
47     C = []
48     for i in range(mid):
49         C.append(C11[i] + C12[i])
50
51     for i in range(mid):
52         C.append(C21[i] + C22[i])
53
54     return C
55
56
57 # Matrix Size
58 N = 8
59
60 A = [[random.randint(1, 10) for _ in range(N)] for _ in range(N)]
61 B = [[random.randint(1, 10) for _ in range(N)] for _ in range(N)]
62
63 for depth in [0,1,2,3]:
64     start = time.perf_counter()
65     result = divide_conquer(A, B, 0, depth)
66     end = time.perf_counter()
67
68     print("\nRecursion Depth =", depth)
69     print("Execution Time =", round((end-start)*1000,4), "ms")
```

OUTPUT

```
Recursion Depth = 0
Execution Time = 0.0497 ms

Recursion Depth = 1
Execution Time = 0.1203 ms

Recursion Depth = 2
Execution Time = 0.2262 ms

Recursion Depth = 3
Execution Time = 0.7073 ms

=== Code Execution Successful ===
```

Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(n^3)$
-----------	----------

Average Case	$O(n^3)$
--------------	----------

Worst Case	$O(n^3)$
------------	----------

Space Complexity

Case	Space Complexity
------	------------------

Best Case	$O(n^2)$
-----------	----------

Average Case	$O(n^2)$
--------------	----------

Worst Case	$O(n^2)$
------------	----------

Result

The Divide and Conquer Matrix Multiplication algorithm was successfully implemented and tested using different recursion depths. The execution time increased as the recursion depth increased because additional recursive calls introduced computational overhead. Shallow recursion (Depth 0 or 1) produced the fastest execution, while deeper recursion (Depth 2 or 3) incurred higher overhead without reducing the overall time complexity. Therefore, a **small recursion depth (around 1)** provides the best balance between recursive decomposition and execution efficiency for this implementation.